

**“AÑO DE LA ESPERANZA Y EL FORTALECIMIENTO DE LA DEMOCRACIA”**

**TÍTULO:**

# SISTEMA DE GESTIÓN DE PROCESOS DE UN SISTEMA OPERATIVO EN C++

**DOCENTE:**

Jaqueline Karol Inga Pérez

**INTEGRANTES:**


1. Rafael Zacarias, Jhovany Nico
2. Porras Benito, Flavio Isac
3. Buendia de la Rosa, Eric Daniel
4. Mayor Marticorena Leonel Eduardo


**CURSO:**

**ESTRUCTURA DE DATOS**

**HUANCAYO**

**202**

# ÍNDICE GENERAL

**1. INTRODUCCIÓN Y ANÁLISIS DEL PROBLEMA 3**
1.1. Contexto 3
1.2. Objetivos del proyecto 3
1.2.1. Objetivo general 3
1.2.2. Objetivos específicos 3
1.3. Requerimientos del sistema 4
1.3.1. Requerimientos funcionales 4
1.3.2. Requerimientos no funcionales 5
**2. DISEÑO DE LA SOLUCIÓN Y ARQUITECTURA 6**
2.1. Arquitectura modular del sistema 6
2.2. Definición de entidades y registros (struct) 6
2.3. Diagramas de las estructuras de datos 7
2.3.1. Lista enlazada simple – Gestor de procesos 7
2.3.2. Cola FIFO – Planificador de CPU 7
2.3.3. Pila – Gestor de memoria 8
2.4. Pseudocódigo de las operaciones principales 8
2.4.1. Inserción al final de la lista enlazada (registrar proceso) 9
2.4.2. Ordenamiento de procesos por prioridad 9
2.4.3. Asignación y liberación de memoria RAM (push y pop) 10
2.4.4. Encolar y ejecutar tareas de CPU 10
2.4.5. Eliminación de nodos en la lista enlazada 10
**3. EXPLICACIÓN DETALLADA DE LAS ESTRUCTURAS DE DATOS Y JUSTIFICACIÓN 12**
3.1. Estructuras estáticas (unidades 1 y 2) 12
3.1.1. Arreglos (arrays) 12
3.1.2. Cadenas y registros (struct) 13
3.2. Estructuras dinámicas lineales (unidad 3) 13
3.2.1. Lista enlazada – Gestor de procesos 13
3.2.2. Cola de CPU (FIFO) – Planificador de CPU 13
3.2.3. Pila – Gestor de memoria 14
3.3. Persistencia de datos (archivos) 14
**4. ANÁLISIS DE ESTRUCTURAS DE CONTROL Y COMPLEJIDAD 16**
4.1. Análisis de estructuras de programación utilizadas 16
4.1.1. Estructuras repetitivas 16
4.1.2. Estructuras condicionales 16
4.2. Análisis teórico de complejidad temporal y espacial 17
**5. MANUAL DE USUARIO Y VALIDACIÓN DE PRUEBAS 18**
5.1. Guía de compilación y ejecución 18
5.2. Guía de operaciones del menú 18
5.3. Evidencias de ejecución (capturas de pantalla) 19

5.3.1. Captura 1. Creación de procesos y asignación exitosa de memoria RAM 20
5.3.2. Captura 2. Búsqueda y modificación de datos o prioridades de procesos 22
5.3.3. Captura 3. Ejecución y despacho en CPU respetando el orden de llegada 25
5.3.4. Captura 4. Manejo de errores y validación de entradas 26
5.3.5. Captura 5. Guardado y recuperación del estado desde archivo 29
**6. GESTIÓN DEL TRABAJO EN EQUIPO Y CONTROL DE VERSIONES 32**
6.1. Evidencia del repositorio Git 32
6.2. Matriz de roles y responsabilidades 32
6.3. Actas de reunión y seguimiento 32
**7. CONCLUSIONES Y RECOMENDACIONES 33**
7.1. Conclusiones 33
7.2. Dificultades y soluciones aplicadas 33
7.3. Recomendaciones 34
**8. REFERENCIAS BIBLIOGRÁFICAS 35**
**9. ANEXOS 36**

# 1. INTRODUCCIÓN Y ANÁLISIS DEL PROBLEMA

## 1.1. Contexto

Un sistema operativo debe administrar de forma simultánea los procesos que se están ejecutando, la memoria RAM que estos ocupan y el orden en que la CPU atiende sus tareas (Silberschatz et al., 2018). Cada una de estas actividades sigue una lógica de acceso distinta: los procesos se registran, consultan y reordenan según su prioridad; la memoria se asigna y se libera desde el bloque más reciente; y las tareas de CPU se atienden en el orden en que llegan.

Para representar estas actividades mediante programación se desarrolló un sistema en C++ dividido en tres módulos. El **Gestor de Procesos** utiliza una lista enlazada simple para registrar, buscar, modificar y ordenar procesos según su prioridad. El **Gestor de Memoria RAM** utiliza una pila LIFO para asignar y liberar bloques de memoria, de modo que el último bloque asignado sea el primero en liberarse. El **Planificador de CPU** utiliza una cola FIFO para encolar y ejecutar tareas en orden de llegada. Además, el sistema incorpora persistencia en archivos de texto, lo que permite conservar los datos entre ejecuciones, y valida cada dato ingresado por el usuario desde la consola.

El problema que resuelve el proyecto es, por tanto, elegir la estructura de datos lineal más adecuada para cada comportamiento del sistema operativo e implementarla desde cero con punteros y memoria dinámica (`new` y `delete`), sin recurrir a contenedores de la biblioteca estándar.

## 1.2. Objetivos del proyecto

### 1.2.1. Objetivo general

Desarrollar un sistema de gestión de procesos de un sistema operativo en C++ aplicando una lista enlazada simple, una pila LIFO y una cola FIFO, con persistencia de datos en archivos de texto, validación de entradas y liberación correcta de la memoria dinámica.

### 1.2.2. Objetivos específicos

- Implementar una lista enlazada simple para registrar, mostrar, buscar, modificar y ordenar procesos por prioridad.
- Implementar una pila LIFO para asignar bloques de memoria RAM, liberarlos y consultar el estado de la memoria.
- Implementar una cola FIFO para encolar tareas de CPU, ejecutarlas y consultar su orden de atención.
- Aplicar registros `struct`, punteros, reserva de memoria con `new` y liberación con `delete`.

- Validar los datos ingresados por el usuario para evitar errores, bloqueos o ciclos infinitos.
- Implementar persistencia con `<fstream>` para guardar y cargar procesos, bloques de memoria y tareas de CPU.
- Organizar el programa mediante funciones y menús que faciliten su uso y mantenimiento.

## 1.3. Requerimientos del sistema

### 1.3.1. Requerimientos funcionales

La Tabla 1 resume los requerimientos funcionales y el módulo del programa que los atiende.

**Tabla 1.** *Requerimientos funcionales del sistema*

| Código | Requerimiento | Módulo / función |
|---|---|---|
| RF-01 | Acceder a los tres módulos y a la opción Salir desde un menú principal. | `main()` |
| RF-02 | Registrar procesos con ID, nombre y prioridad, sin IDs repetidos. | Procesos · `registrarProceso()` |
| RF-03 | Mostrar todos los procesos registrados y su total. | Procesos · `mostrarProcesos()` |
| RF-04 | Buscar un proceso por ID. | Procesos · `consultarProceso()` |
| RF-05 | Modificar la prioridad de un proceso. | Procesos · `modificarPrioridadProceso()` |
| RF-06 | Ordenar los procesos por prioridad de mayor a menor. | Procesos · `ordenarProcesosPorPrioridad()` |
| RF-07 | Asignar bloques de memoria (ID y tamaño en MB) mediante push. | Memoria · `asignarMemoria()` |
| RF-08 | Liberar el bloque ubicado en el tope mediante pop. | Memoria · `liberarMemoria()` |
| RF-09 | Mostrar el estado de la pila de memoria marcando el tope. | Memoria · `mostrarEstadoMemoria()` |
| RF-10 | Buscar un bloque por ID e indicar su posición desde el tope. | Memoria · `consultarBloqueMemoria()` |
| RF-11 | Encolar tareas de CPU con ID de tarea, ID de proceso y descripción. | CPU · `encolarTarea()` |
| RF-12 | Ejecutar (desencolar) la tarea ubicada en el frente. | CPU · `ejecutarTarea()` |
| RF-13 | Mostrar la cola de tareas pendientes marcando la siguiente. | CPU · `mostrarColaCPU()` |
| RF-14 | Modificar la descripción de una tarea por ID de proceso. | CPU · `modificarTarea()` |

| Código | Requerimiento | Módulo / función |
|---|---|---|
| RF-15 | Buscar una tarea por ID e indicar cuántas tareas la preceden. | CPU · `consultarTarea()` |
| RF-16 | Guardar y cargar el estado en `procesos.txt`, `memoria.txt` y `tareas_cpu.txt`. | Persistencia · `guardar*()` / `cargar*()` |
| RF-17 | Liberar toda la memoria dinámica al salir del programa. | `liberarProcesos()`, `liberarPilaMemoria()`, `liberarColaCPU()` |

### 1.3.2. Requerimientos no funcionales

**Tabla 2.** *Requerimientos no funcionales del sistema*

| Código | Requerimiento no funcional |
|---|---|
| RNF-01 | El programa está desarrollado en C++ y compila en Dev-C++ (compilador GCC/TDM-GCC). |
| RNF-02 | Usa exclusivamente estructuras dinámicas lineales implementadas desde cero: lista enlazada, pila y cola. |
| RNF-03 | Emplea punteros y memoria dinámica mediante `new` y `delete`. |
| RNF-04 | Presenta una interfaz por consola clara y organizada mediante menús y submenús. |
| RNF-05 | Valida datos numéricos y de texto para evitar errores de entrada y ciclos infinitos. |
| RNF-06 | Implementa persistencia de datos mediante archivos de texto con `<fstream>`. |
| RNF-07 | El código está organizado por módulos y funciones para facilitar su mantenimiento. |
| RNF-08 | La memoria reservada dinámicamente se libera al salir del programa (sin fugas de memoria). |

# 2. DISEÑO DE LA SOLUCIÓN Y ARQUITECTURA

## 2.1. Arquitectura modular del sistema

El sistema utiliza una arquitectura modular compuesta por un menú principal y tres módulos especializados (Figura 1). Al iniciar el programa, `main()` carga los datos guardados en los archivos de texto; luego muestra el menú principal dentro de un ciclo `do-while`, y una sentencia `switch` dirige al usuario hacia el submenú elegido. Durante la ejecución, cada operación que modifica una estructura guarda inmediatamente los cambios en su archivo. Al elegir **Salir**, se guardan nuevamente los tres archivos y se libera toda la memoria dinámica.

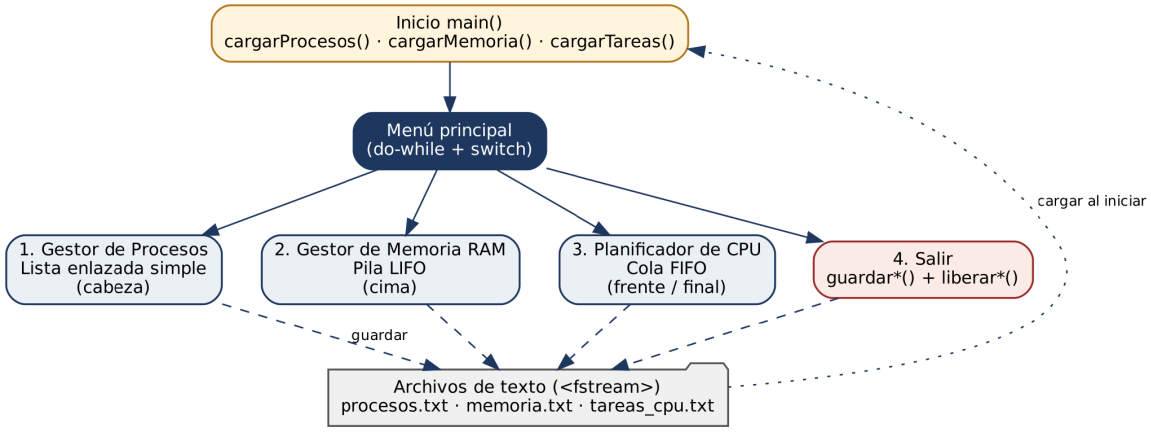



**Figura 1.** *Arquitectura modular del sistema y flujo de persistencia.*

*Nota.* Elaboración propia a partir de la función `main()` (

Definición de entidades y registros (struct)

El sistema separa los datos de cada entidad de los nodos de las estructuras dinámicas: cada nodo contiene un registro completo (`dato`) y un puntero al nodo siguiente. Esta separación permite reutilizar el registro en funciones de lectura, búsqueda y persistencia sin depender de los enlaces. La Tabla 3 describe los siete registros definidos entre las líneas 46 y 84.

**Tabla 3.** *Registros y nodos definidos en el programa*

| Registro | Campos principales | Líneas | Finalidad |
|---|---|---|---|
| `Proceso` | `idProceso` (int), `nombre` (char[30]), `prioridad` (int) | 47–51 | Datos de un proceso del sistema operativo. |
| `NodoLista` | `dato` (Proceso), `siguiente` (NodoLista*) | 53–56 | Nodo de la lista enlazada del Gestor de Procesos. |
| `BloqueMemoria` | `idBloque` (int), `tamanoMB` (int) | 59–62 | Segmento de memoria RAM asignado. |
| `NodoPila` | `dato` (BloqueMemoria), `siguiente` (NodoPila*) | 64–67 | Nodo de la pila LIFO de memoria. |

| Registro | Campos principales | Líneas | Finalidad |
|---|---|---|---|
| `TareaCPU` | `idTarea` (int), `idProceso` (int), `descripcion` (char[50]) | 70–74 | Tarea enviada al procesador. |
| `NodoCola` | `dato` (TareaCPU), `siguiente` (NodoCola*) | 76–79 | Nodo de la cola FIFO de CPU. |
| `PlanificadorCPU` | `frente` (NodoCola), `final` (NodoCola) | 81–84 | Agrupa los dos punteros que controlan la cola. |

## 2.2. Diagramas de las estructuras de datos

Las siguientes figuras representan el estado de cada estructura con los datos de prueba usados en la sección 5.3. Permiten ver cómo se enlazan los nodos y por qué las operaciones de inserción, extracción y recorrido funcionan de manera distinta en cada estructura.

### 2.2.1. Lista enlazada simple – Gestor de procesos

El puntero `cabeza` apunta al primer proceso y cada nodo apunta al siguiente hasta llegar a `NULL`. La inserción se realiza al final: se recorre la lista desde la cabeza hasta el último nodo y se enlaza el nuevo proceso. La búsqueda, la modificación y el ordenamiento también recorren la lista desde la cabeza.

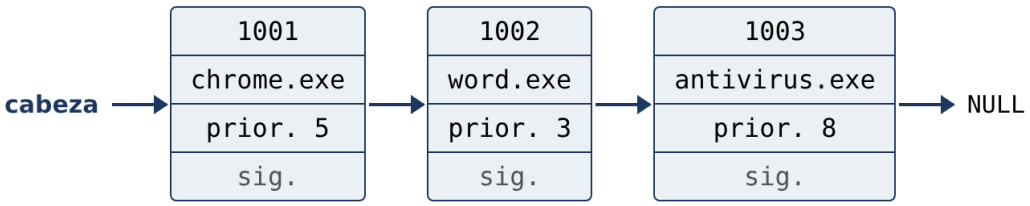



**Figura 2.** *Lista enlazada simple con tres procesos registrados.*

*Nota.* Orden de inserción: 1001, 1002 y 1003. Cada nodo almacena un `Proceso` y el puntero `siguiente`.

### 2.2.2. Cola FIFO – Planificador de CPU

La cola mantiene dos punteros: `frente`, por donde se ejecutan las tareas, y `final`, por donde se encolan. Al encolar, el nuevo nodo se enlaza después del último y `final` se actualiza; al ejecutar, se retira el nodo del frente y `frente` avanza. Si la cola queda vacía, `final` también se asigna a `NULL`. Así, la primera tarea en llegar es la primera en ejecutarse.

**Sobre la prioridad:** el planificador implementado atiende las tareas por orden de llegada (FIFO). La prioridad de los procesos se administra en el Gestor de Procesos y se aplica con `ordenarProcesosPorPrioridad()` (líneas 307–328). Si se requiriera una cola de prioridad estricta, el cambio se concentraría en `encolarTarea()` (líneas 449–481), insertando el nodo según la

prioridad del proceso asociado; ante prioridades iguales, el nuevo nodo se ubicaría detrás de los existentes para conservar el orden de llegada.

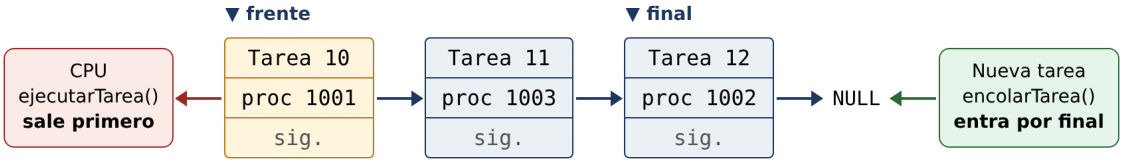



**Figura 3.** *Cola FIFO del Planificador de CPU con tres tareas pendientes.*

### 2.2.3. Pila – Gestor de memoria

La pila se administra con el puntero `cima`. Al asignar memoria (push), el nuevo bloque se enlaza sobre la cima anterior y la cima pasa a ser el nuevo nodo. Al liberar memoria (pop), se retira el bloque de la cima y el puntero avanza al bloque inferior. No se utiliza pila auxiliar: la búsqueda por ID recorre la pila sin modificarla e informa cuántos bloques habría que liberar antes de llegar al bloque buscado.

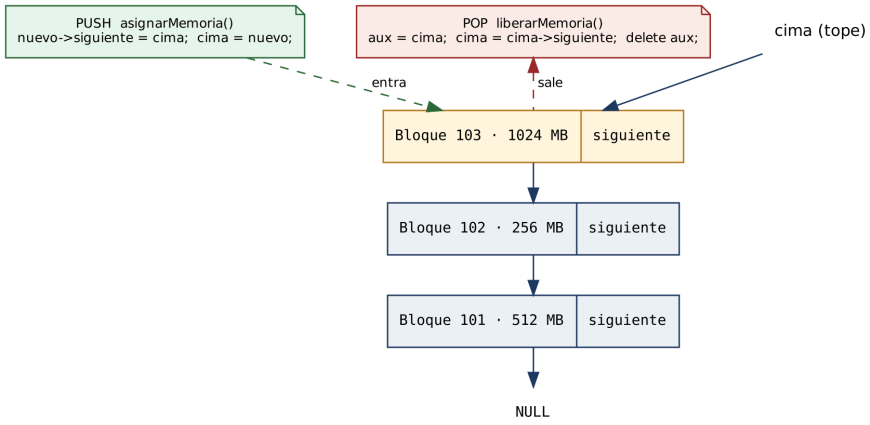



**Figura 4.** *Pila LIFO de memoria RAM después de tres operaciones push.*

*Nota.* El bloque 103 fue el último en asignarse, por eso ocupa la cima y será el primero en liberarse.

## 2.3. Pseudocódigo de las operaciones principales

El pseudocódigo siguiente refleja la lógica implementada en el código fuente. Todas las operaciones de inserción validan los datos en variables locales **antes** de reservar memoria con `new`, de modo que nunca se crea un nodo con datos inválidos.

### 2.3.1. Inserción al final de la lista enlazada (registrar proceso)

```
PROCEDIMIENTO registrarProceso(cabeza)
    LEER idProceso
    SI idProceso no es numérico O idProceso <= 0 ENTONCES
        MOSTRAR "ID inválido";  TERMINAR
    SI buscarProceso(cabeza, idProceso) <> NULL ENTONCES
        MOSTRAR "Ya existe un proceso con ese ID";  TERMINAR
    LEER nombre
    SI nombre está vacío ENTONCES MOSTRAR "No se registró";  TERMINAR
    LEER prioridad
    SI prioridad no es numérica O prioridad <= 0 ENTONCES
        MOSTRAR "Prioridad inválida";  TERMINAR
    nuevo <- new NodoLista
    nuevo.dato <- (idProceso, nombre, prioridad);  nuevo.siguiente <- NULL
    SI cabeza = NULL ENTONCES cabeza <- nuevo
    SINO
        actual <- cabeza
        MIENTRAS actual.siguiente <> NULL HACER actual <- actual.siguiente
        actual.siguiente <- nuevo
    guardarProcesos(cabeza)
FIN PROCEDIMIENTO
```

**Figura 5.** *Pseudocódigo de `registrarProceso()` (líneas 212–247).*

### 2.3.2. Ordenamiento de procesos por prioridad

```
PROCEDIMIENTO ordenarProcesosPorPrioridad(cabeza)
    SI cabeza = NULL O cabeza.siguiente = NULL ENTONCES TERMINAR
    REPETIR
        intercambio <- FALSO;  actual <- cabeza
        MIENTRAS actual.siguiente <> NULL HACER
            SI actual.dato.prioridad < actual.siguiente.dato.prioridad ENTONCES
                intercambiar(actual.dato, actual.siguiente.dato)
                intercambio <- VERDADERO
            actual <- actual.siguiente
    HASTA QUE intercambio = FALSO
    guardarProcesos(cabeza)
FIN PROCEDIMIENTO
```

**Figura 6.** *Pseudocódigo del ordenamiento burbuja por prioridad (líneas 307–328).*

*Nota.* Se intercambian los datos de los nodos, no los enlaces, por lo que la cabeza de la lista no cambia de dirección.

### 2.3.3. Asignación y liberación de memoria RAM (push y pop)

```
PROCEDIMIENTO asignarMemoria(cima)                // PUSH
    LEER idBloque;  validar numérico, > 0 y no repetido
    LEER tamanoMB;  validar numérico y > 0
    nuevo <- new NodoPila
    nuevo.dato <- (idBloque, tamanoMB)
    nuevo.siguiente <- cima
    cima <- nuevo
    guardarMemoria(cima)
FIN PROCEDIMIENTO

PROCEDIMIENTO liberarMemoria(cima)                // POP
    SI cima = NULL ENTONCES MOSTRAR "Pila vacía";  TERMINAR
    aux <- cima;  MOSTRAR aux.dato
    cima <- cima.siguiente
    delete aux
    guardarMemoria(cima)
FIN PROCEDIMIENTO
```

**Figura 7.** *Pseudocódigo de push (líneas 353–376) y pop (líneas 378–390).*

### 2.3.4. Encolar y ejecutar tareas de CPU

```
PROCEDIMIENTO encolarTarea(cpu)
    LEER idTarea;  validar numérico, > 0 y no repetido
    LEER idProceso;  validar numérico y > 0
    LEER descripcion;  validar que no esté vacía
    nuevo <- new NodoCola;  nuevo.dato <- (idTarea, idProceso, descripcion)
    nuevo.siguiente <- NULL
    SI cpu.frente = NULL ENTONCES cpu.frente <- nuevo
    SINO cpu.final.siguiente <- nuevo
    cpu.final <- nuevo
    guardarTareas(cpu)
FIN PROCEDIMIENTO

PROCEDIMIENTO ejecutarTarea(cpu)
    SI cpu.frente = NULL ENTONCES MOSTRAR "No hay tareas";  TERMINAR
    aux <- cpu.frente;  MOSTRAR aux.dato
    cpu.frente <- cpu.frente.siguiente
    SI cpu.frente = NULL ENTONCES cpu.final <- NULL
    delete aux
    guardarTareas(cpu)
FIN PROCEDIMIENTO
```

**Figura 8.** *Pseudocódigo de encolar (líneas 449–481) y ejecutar (líneas 483–500).*

### 2.3.5. Eliminación de nodos en la lista enlazada

En la versión actual, la eliminación de nodos de la lista se realiza al salir del programa con `liberarProcesos()` (líneas 330–336), que elimina siempre el primer nodo hasta vaciar la lista. La eliminación individual de un proceso (al inicio, en medio o al final) no forma parte de esta versión; su pseudocódigo se propone como mejora en la sección 7.3.

```
PROCEDIMIENTO liberarProcesos(cabeza)        // implementado
    MIENTRAS cabeza <> NULL HACER
        aux <- cabeza
        cabeza <- cabeza.siguiente
        delete aux
FIN PROCEDIMIENTO

PROCEDIMIENTO eliminarProceso(cabeza, id)    // propuesta de mejora
    SI cabeza = NULL ENTONCES TERMINAR
    SI cabeza.dato.idProceso = id ENTONCES          // al inicio
        aux <- cabeza;  cabeza <- cabeza.siguiente;  delete aux;  TERMINAR
    anterior <- cabeza;  actual <- cabeza.siguiente
    MIENTRAS actual <> NULL Y actual.dato.idProceso <> id HACER
        anterior <- actual;  actual <- actual.siguiente
    SI actual <> NULL ENTONCES                      // en medio o al final
        anterior.siguiente <- actual.siguiente
        delete actual
FIN PROCEDIMIENTO
```

**Figura 9.** *Liberación de la lista (implementada) y eliminación individual (propuesta).*

# 3. EXPLICACIÓN DETALLADA DE LAS ESTRUCTURAS DE DATOS Y JUSTIFICACIÓN

## 3.1. Estructuras estáticas (unidades 1 y 2)

### 3.1.1. Arreglos (arrays)

El sistema utiliza arreglos unidimensionales de tipo `char` para almacenar textos acotados: `char nombre[30]` en el registro `Proceso` y `char descripcion[50]` en `TareaCPU`. Al tener un tamaño fijo conocido, cada registro ocupa siempre la misma cantidad de memoria. La lectura se realiza con `cin.getline()` dentro de `leerTexto()`, que además controla el desbordamiento: si el texto supera el tamaño del arreglo, limpia el estado de error, descarta el sobrante y avisa al usuario que se guardaron solo los primeros caracteres. La copia de cadenas se realiza con `strcpy()` y la longitud se verifica con `strlen()` (biblioteca `<cstring>`).

```
// ================= VALIDACIÓN Y LECTURA DE DATOS =================

bool entradaValida() {
    if (cin.fail()) {
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
        return false;
    }
    return true;
}

bool leerEntero(const char mensaje[], int &valor) {
    cout << mensaje;
    cin >> valor;
    if (!entradaValida()) {
        return false;
    }
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
    return true;
}

bool leerTexto(const char mensaje[], char destino[], int tamano) {
    cout << mensaje;
    cin.getline(destino, tamano);
    if (cin.fail()) {
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
        cout << "(Texto muy largo: se guardaron los primeros "
             << tamano - 1 << " caracteres)\n";
    }
    if (strlen(destino) == 0) {
        cout << "\nEl texto no puede estar vacio.\n";
        return false;
    }
    return true;
}
```

**Figura 10.** *Funciones de validación y lectura de datos (líneas 9–42).*

*Nota.* `entradaValida()` evita ciclos infinitos cuando se ingresan letras en campos numéricos; `leerTexto()` controla textos vacíos o demasiado largos.

### 3.1.2. Cadenas y registros (struct)

Los registros `struct` agrupan datos relacionados: `Proceso` agrupa ID, nombre y prioridad; `BloqueMemoria` agrupa ID y tamaño; `TareaCPU` agrupa ID de tarea, ID de proceso y descripción. Se emplean estructuras anidadas (por ejemplo, `NodoLista` contiene un `Proceso`) para separar los datos del dominio de los punteros de enlace. Esto facilita el paso de datos entre funciones y permite que cada nodo almacena una entidad completa.

## 3.2. Estructuras dinámicas lineales (unidad 3)

### 3.2.1. Lista enlazada – Gestor de procesos

La lista enlazada simple es adecuada para el Gestor de Procesos porque la cantidad de procesos de un sistema operativo cambia constantemente y no conviene fijar un tamaño máximo, como ocurriría con un arreglo (Joyanes Aguilar y Zahonero Martínez, 2007). Además, la lista permite recorrer los procesos de forma secuencial para buscarlos, modificarlos y ordenarlos por prioridad. Las operaciones de búsqueda e inserción al final son O(n), porque requieren recorrer la lista desde la cabeza.

```
201 NodoLista* buscarProceso(NodoLista* cabeza, int idBuscado) {
202     NodoLista* actual = cabeza;
203     while (actual != NULL) {
204         if (actual->dato.idProceso == idBuscado) {
205             return actual;
206         }
207         actual = actual->siguiente;
208     }
209     return NULL;
210 }
```

```
231     NodoLista* nuevo = new NodoLista;
232     nuevo->dato = nuevoDato;
233     nuevo->siguiente = NULL;
234
235     if (cabeza == NULL) {
236         cabeza = nuevo;
237     } else {
238         NodoLista* actual = cabeza;
239         while (actual->siguiente != NULL) {
240             actual = actual->siguiente;
241         }
242         actual->siguiente = nuevo;
243     }
```

**Figura 11.** *Búsqueda e inserción al final en la lista enlazada (líneas 201–210 y 231–243).*

### 3.2.2. Cola de CPU (FIFO) – Planificador de CPU

La cola FIFO es adecuada para el planificador porque garantiza un trato equitativo: las tareas se ejecutan en el mismo orden en que llegan, como en la planificación *First-Come, First-Served* (Silberschatz et al., 2018). Al mantener los punteros `frente` y `final`, el enlace de una nueva tarea y la extracción de la siguiente se realizan en O(1), sin recorrer la cola. La validación de IDs repetidos sí recorre la cola, por lo que el registro completo de una tarea es O(n).

```
468     NodoCola* nuevo = new NodoCola;
469     nuevo->dato = nuevoDato;
470     nuevo->siguiente = NULL;
471
472     if (cpu.frente == NULL) {
473         cpu.frente = nuevo;
474     } else {
475         cpu.final->siguiente = nuevo;
476     }
477     cpu.final = nuevo;
```

```
494     cpu.frente = cpu.frente->siguiente;
495     if (cpu.frente == NULL) {
496         cpu.final = NULL;
497     }
498     delete aux;
```

**Figura 12.** *Enlace al final (encolar) y extracción del frente (ejecutar) en la cola de CPU.*

### 3.2.3. Pila – Gestor de memoria

La pila representa de forma natural la asignación de memoria por bloques: el último bloque asignado se encuentra en el tope y es el primero en liberarse, de manera similar a como se apilan los marcos de función en la pila de ejecución de un programa (Cairó y Guardati, 2006). Las operaciones push y pop trabajan únicamente sobre la cima, por lo que su enlace es O(1). La búsqueda por ID requiere recorrer la pila (O(n)) e informa la posición del bloque desde el tope.

```
368
369         NodoPila* nuevo = new NodoPila;
370         nuevo->dato = nuevoDato;
371         nuevo->siguiente = cima;
372         cima = nuevo;
373
```

```
383     NodoPila* aux = cima;
384     cout << "\nBloque de memoria liberado del tope (Pop) -> ID: " << aux->dato.idBloque
385          << " | Tamano: " << aux->dato.tamanoMB << " MB\n";
386
387     cima = cima->siguiente;
388     delete aux;
389     guardarMemoria(cima);
390 }
```

**Figura 13.** *Enlace de un nodo (push) y liberación del tope (pop) en la pila de memoria.*

### 3.3. Persistencia de datos (archivos)

El sistema implementa persistencia con la biblioteca `<fstream>`. Cada módulo tiene su propio archivo: los procesos se guardan en `procesos.txt`, los bloques de memoria en `memoria.txt` y las tareas de CPU en `tareas_cpu.txt`. El formato es de un campo por línea, lo que permite almacenar textos con espacios.

- **Guardado:** `guardarProcesos()`, `guardarMemoria()` y `guardarTareas()` sobrescriben el archivo recorriendo la estructura de principio a fin. Se invocan después de cada operación que modifica datos y nuevamente al salir.
- **Carga:** `cargarProcesos()`, `cargarMemoria()` y `cargarTareas()` se ejecutan al iniciar `main()`. Leen el archivo campo por campo, crean cada nodo con `new` y lo enlazan **al final** para conservar el orden original (en la pila, el primer bloque leído vuelve a ser la cima).
- **Control de errores:** si un archivo no existe, `is_open()` devuelve falso y el sistema continúa con la estructura vacía. Se usa `ignore()` para descartar el salto de línea

que deja `>>` antes de leer textos con `getline()`.

```
176 void cargarTareas(PlanificadorCPU& cpu) {
177     ifstream archivo("tareas_cpu.txt");
178     if (!archivo.is_open()) return;
179     TareaCPU t;
180     while (archivo >> t.idTarea) {
181         archivo >> t.idProceso;
182         archivo.ignore(numeric_limits<streamsize>::max(), '\n');
183         archivo.getline(t.descripcion, 50);
184 
185         NodoCola* nuevo = new NodoCola;
186         nuevo->dato = t;
187         nuevo->siguiente = NULL;
188 
189         if (cpu.frente == NULL) {
190             cpu.frente = nuevo;
191         } else {
192             cpu.final->siguiente = nuevo;
193         }
194         cpu.final = nuevo;
195     }
196     archivo.close();
197 }
```

**Figura 14.** *Carga de la cola de CPU desde `tareas_cpu.txt` (líneas 176–197).*

```
674     // Carga de la persistencia guardada previamente
675     cargarProcesos(procesos);
676     cargarMemoria(memoria);
677     cargarTareas(cpu);
678
679     int opcion;
680
681     do {
682         cout << "\n=============================================\n";
683         cout << "  SISTEMA DE GESTION DE PROCESOS - SO\n";
684         cout << "=============================================\n";
685         cout << "1. Gestor de Procesos (Lista Enlazada)\n";
686         cout << "2. Gestor de Memoria RAM (Pila LIFO)\n";
687         cout << "3. Planificador de CPU (Cola FIFO)\n";
688         cout << "4. Salir del programa\n";
689
690         if (!leerEntero("Seleccione una opcion: ", opcion)) {
691             opcion = 0;
692         }
693
694         switch (opcion) {
695             case 1: menuProcesos(procesos); break;
696             case 2: menuMemoria(memoria); break;
697             case 3: menuCPU(cpu); break;
698             case 4:
699                 guardarProcesos(procesos);
700                 guardarMemoria(memoria);
701                 guardarTareas(cpu);
702                 liberarProcesos(procesos);
703                 liberarPilaMemoria(memoria);
704                 liberarColaCPU(cpu);
705                 cout << "\nDatos guardados en disco y memoria liberada correctamente.\n";
706                 cout << "Programa finalizado.\n";
707                 break;
708             default:
709                 cout << "\nOpcion no valida. Intente de nuevo.\n";
710     }
```

**Figura 15.** *Carga inicial y guardado final con liberación de memoria en `main()`.*

# 4. ANÁLISIS DE ESTRUCTURAS DE CONTROL Y COMPLEJIDAD

## 4.1. Análisis de estructuras de programación utilizadas

El programa utiliza estructuras repetitivas y condicionales para controlar el flujo, validar datos y recorrer las estructuras dinámicas. El análisis de eficiencia se realiza de forma teórica (notación O grande), sin medir tiempos reales de ejecución.

### 4.1.1. Estructuras repetitivas

- `do-while`: se utiliza en los cuatro menús para mostrar las opciones al menos una vez y repetirlas hasta que el usuario elija salir. También controla las pasadas del ordenamiento burbuja mientras existan intercambios.
- `while`: recorre los nodos de la lista, la pila y la cola hasta llegar a `NULL`; lee los archivos mientras existan datos (`while (archivo >> t.idTarea)`), y libera la memoria nodo por nodo.
- `for`: no se utiliza, porque la cantidad de nodos no se conoce de antemano; los recorridos terminan al encontrar `NULL`.

### 4.1.2. Estructuras condicionales

Las sentencias `if` validan que los datos sean correctos antes de reservar memoria (ID numérico y mayor que cero, ID no repetido, texto no vacío) y verifican si una estructura está vacía antes de extraer un elemento (por ejemplo, `if (cima == NULL)`). La sentencia `switch` dirige la ejecución a la opción elegida en cada menú, y su caso `default` informa cuando la opción no es válida. La Tabla 4 resume dónde se usa cada estructura.

**Tabla 4.** *Estructuras de control utilizadas en el programa*

| Estructura | Dónde se usa | Líneas | Qué hace |
|---|---|---|---|
| `do-while` | Menús y ordenamiento burbuja | 587–610, 615–636, 641–664, 681–711, 313–325 | Repite hasta elegir salir o hasta que no haya intercambios. |
| `while` | Recorridos de lista, pila y cola | 203–208, 257–264, 343–349, 400–407, 440–445, 510–518, 561–570 | Recorre los nodos hasta `NULL`. |
| `while` | Lectura de archivos | 105, 145, 180 | Lee mientras existan registros. |
| `while` | Liberación de memoria | 331–335, 429–433, 575–579 | Elimina nodos con `delete`. |
| `if` | Validaciones y estructuras vacías | 10–14, 37–40, 214–229, 356–367, 379–382, 451–466, 484–487 | Evita errores y reservas innecesarias. |

| Estructura | Dónde se usa | Líneas | Qué hace |
|---|---|---|---|
| `switch` | Menús | 601–609, 628–635, 655–663, 694–710 | Dirige a cada opción; `default` maneja opciones inválidas. |

## 4.2. Análisis teórico de complejidad temporal y espacial

La complejidad temporal indica cómo crece el número de operaciones según la cantidad de elementos *n*; la complejidad espacial indica la memoria adicional utilizada, sin contar los nodos que ya almacena la estructura (Cormen et al., 2009). En el sistema, las operaciones sobre el tope de la pila y el frente de la cola son O(1), los recorridos son O(n) y el ordenamiento burbuja es $O(n^2)$ en el peor caso.

**Tabla 5.** *Complejidad temporal y espacial de las operaciones principales*

| Operación | Tiempo | Espacio | Justificación |
|---|---|---|---|
| Buscar proceso / bloque / tarea | O(n) | O(1) | Recorre la estructura con un puntero auxiliar hasta encontrar el ID. |
| Registrar proceso | O(n) | O(1) | Válida duplicados y recorre hasta el último nodo para enlazar. |
| Modificar prioridad | O(n) | O(1) | Busca por ID y cambia el dato en el nodo encontrado. |
| Ordenar por prioridad | $O(n^2)$ peor caso<br>O(n) mejor caso | O(1) | Burbuja con intercambio de datos; si ya está ordenada, termina en una pasada. |
| Push memoria | O(n)* | O(1) | Válida duplicados (O(n)); el enlace en la cima es O(1). |
| Pop memoria | O(1) | O(1) | Retira la cima y actualiza un puntero. |
| Encolar tarea | O(n)* | O(1) | Válida duplicados (O(n)); el enlace con `final` es O(1). |
| Ejecutar tarea | O(1) | O(1) | Retira el nodo del frente. |
| Mostrar lista / pila / cola | O(n) | O(1) | Recorre todos los nodos. |
| Guardar / cargar archivos | O(n) | O(1) | Recorre la estructura o el archivo una sola vez. |

*Nota.* (*) Sin la validación de IDs repetidos, push y encolar serían O(1). Es una decisión de diseño: se acepta un recorrido adicional para garantizar la integridad de los datos.

# 5. MANUAL DE USUARIO Y VALIDACIÓN DE PRUEBAS

## 5.1. Guía de compilación y ejecución

- Abrir Dev-C++ y seleccionar **Archivo > Abrir proyecto o archivo**; elegir `Semana_12_Final.cpp`.
- Verificar el compilador en **Herramientas > Opciones del compilador** (GCC/TDM-GCC incluido con Dev-C++). No se requieren bibliotecas externas: el programa usa solo `<iostream>`, `<fstream>`, `<cstring>` y `<limits>`.
- Presionar **Compilar y ejecutar (F11)**. El código compila con 0 errores y 0 advertencias (también se verificó con GCC usando las opciones `-Wall -Wextra`).
- Se abre una consola con el menú principal (Figura 16). Los archivos `procesos.txt`, `memoria.txt` y `tareas_cpu.txt` se crean en la misma carpeta del ejecutable al registrar datos o al elegir **Salir**.
- Si aparece el error de archivo en uso al recompilar, cerrar la consola del programa anterior y volver a presionar F11.

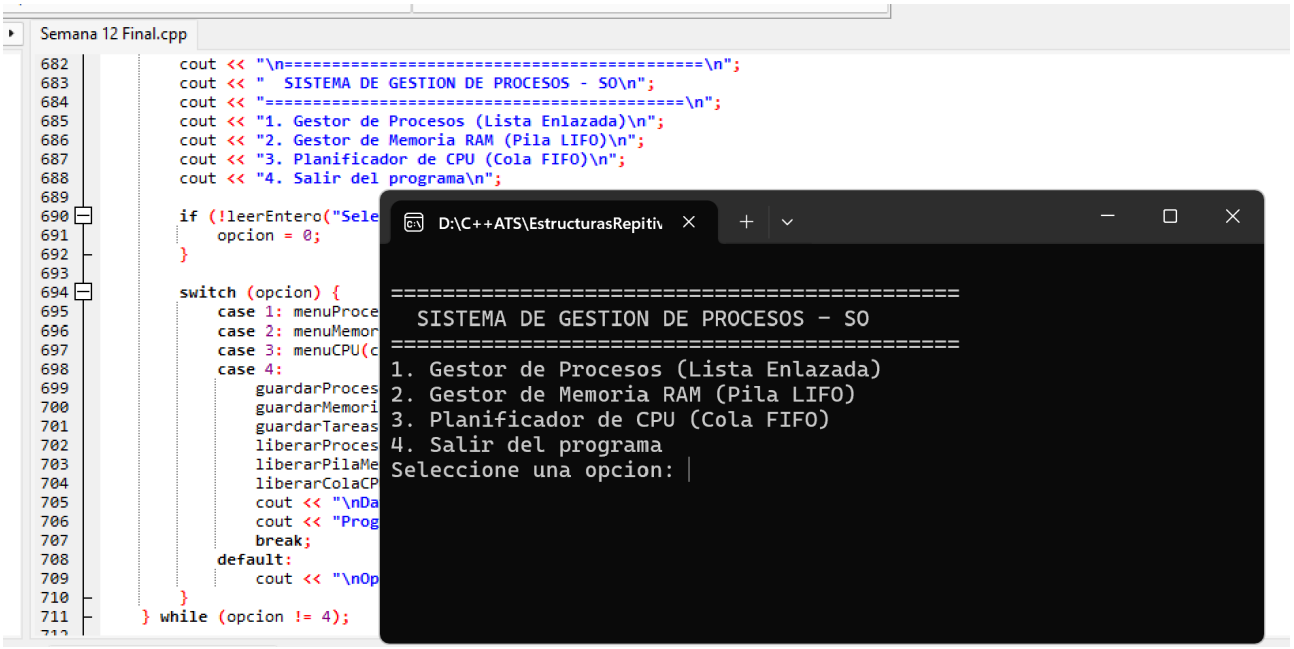



**Figura 16.** *Menú principal del sistema al iniciar el programa.*

## 5.2. Guía de operaciones del menú

El menú principal ofrece cuatro opciones: Gestor de Procesos, Gestor de Memoria RAM, Planificador de CPU y Salir. Cada módulo tiene su propio submenú. Todas las entradas numéricas se validan; si el usuario ingresa letras o valores fuera de rango, se muestra un mensaje y no se reserva memoria.

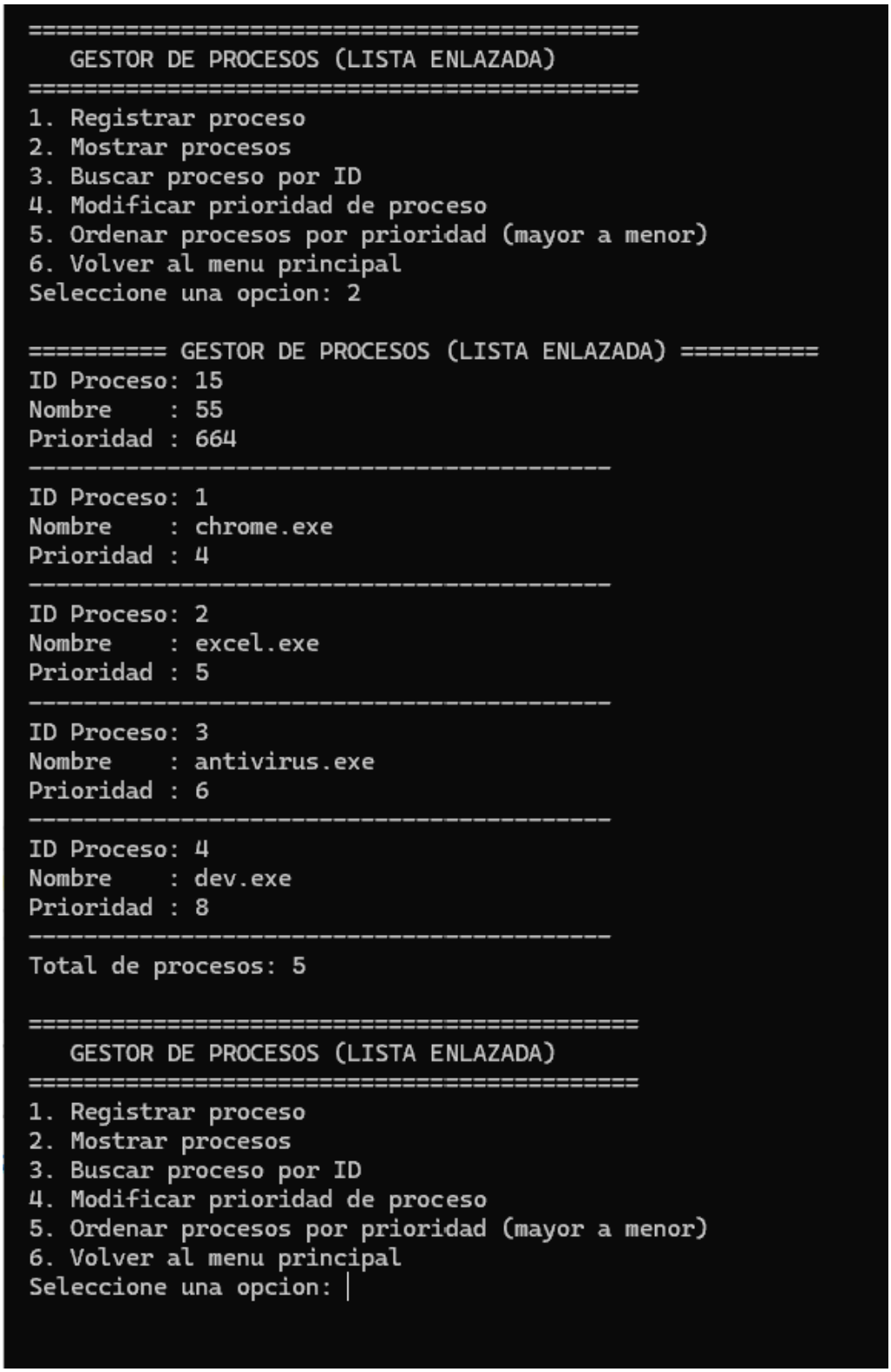
**Tabla 6.** *Guía de operaciones de los menús*

| Opción del menú | Datos solicitados | Acción realizada | Resultado esperado |
|---|---|---|---|
| 1.1 Registrar proceso | ID, nombre, prioridad | Inserta al final de la lista | "Proceso registrado correctamente en la lista." |
| 1.2 Mostrar procesos | — | Recorre la lista | Listado de procesos y total. |
| 1.3 Buscar proceso | ID | Búsqueda secuencial | "Proceso encontrado" o "Proceso no encontrado." |
| 1.4 Modificar prioridad | ID, nueva prioridad | Cambia la prioridad del nodo | "Prioridad actualizada correctamente." |
| 1.5 Ordenar por prioridad | — | Burbuja de mayor a menor | "Procesos ordenados por prioridad." |
| 2.1 Asignar bloque (Push) | ID de bloque, tamaño en MB | Apila el bloque en la cima | "Bloque ... asignado en el tope de la RAM (Push)." |
| 2.2 Liberar bloque (Pop) | — | Desapila la cima | Muestra el bloque liberado. |
| 2.3 Mostrar memoria | — | Recorre la pila | Listado con la marca `[TOPE RAM]`. |
| 2.4 Buscar bloque | ID de bloque | Recorre la pila | Posición desde el tope y bloques por liberar antes. |
| 3.1 Encolar tarea | ID de tarea, ID de proceso, descripción | Enlaza al final de la cola | "Tarea registrada en la cola ... (Encolar)." |
| 3.2 Ejecutar tarea | — | Desencola el frente | Muestra la tarea ejecutada. |
| 3.3 Mostrar cola | — | Recorre la cola | Listado con la marca `[SIGUIENTE EN CPU]`. |
| 3.4 Modificar tarea | ID de proceso, nueva descripción | Cambia la descripción de la primera tarea de ese proceso | "Tarea actualizada correctamente." |
| 3.5 Buscar tarea | ID de tarea | Recorre la cola | Tareas por ejecutar antes de ella. |
| 4 Salir | — | Guarda los tres archivos y libera la memoria | "Datos guardados en disco y memoria liberada correctamente." |

## 5.3. Evidencias de ejecución (capturas de pantalla)

Las capturas corresponden a una secuencia de pruebas real del programa, realizada con datos coherentes con un sistema operativo (procesos `chrome.exe`, `word.exe` y `antivirus.exe`; bloques 101, 102 y 103; tareas "Abrir navegador", "Actualizar antivirus" y "Guardar documento"). Los datos ingresados por el usuario se muestran en amarillo. Para no repetir el mismo texto, los menús que vuelven a mostrarse se resumen con una línea en verde.

### 5.3.1. Captura 1. Creación de procesos y asignación exitosa de memoria RAM



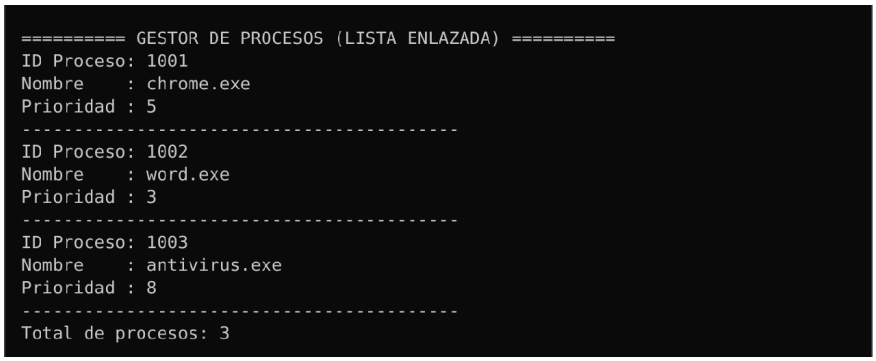



**Figura 17.** *Registro y listado de procesos en la lista enlazada.*

*Nota.* Los procesos se muestran en el orden de inserción, lo que confirma la inserción al final de la lista.

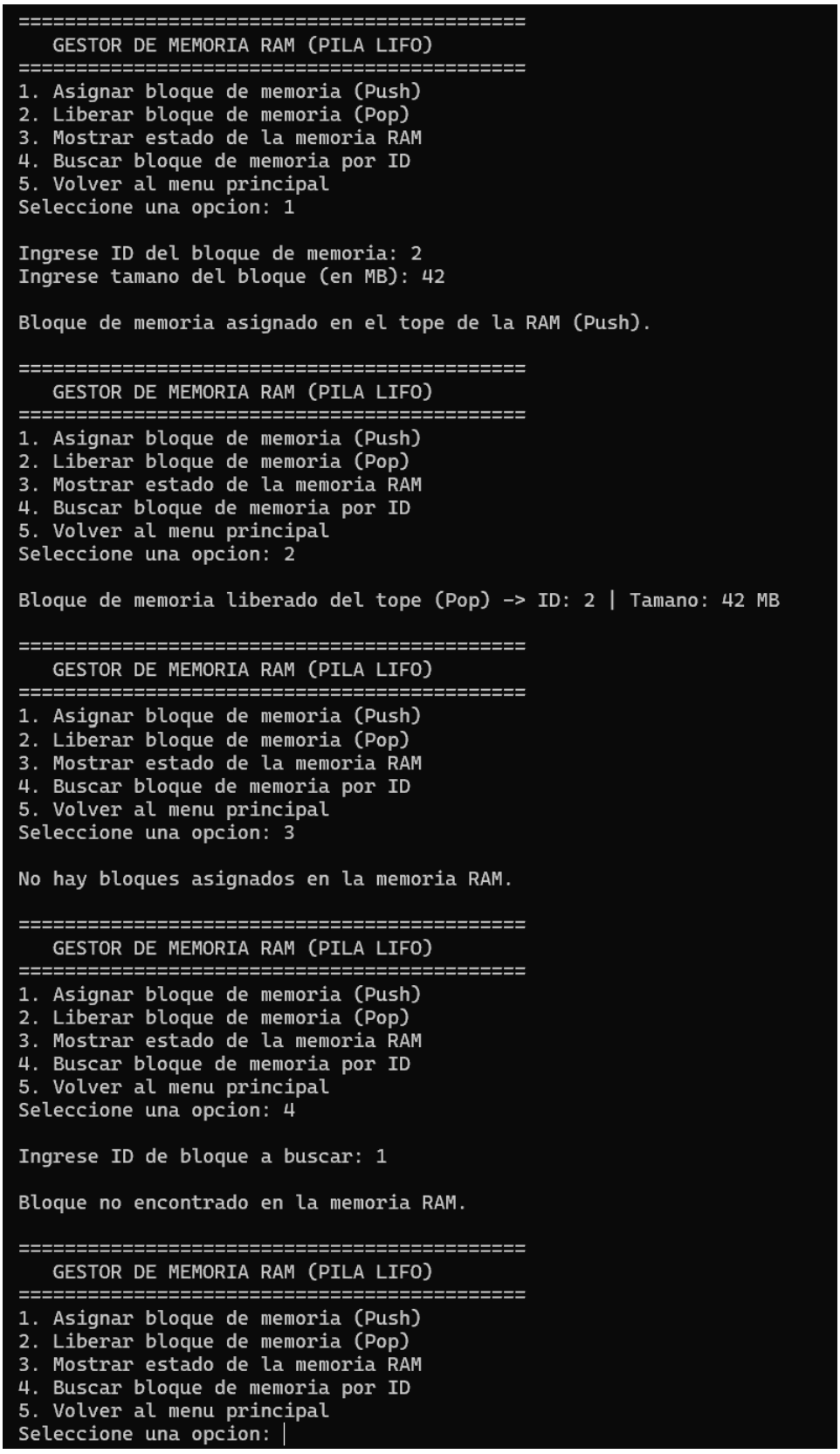



**Figura 18.** *Asignación de tres bloques de memoria (push) y estado de la pila.*

*Nota.* El bloque 103, asignado al último, aparece en la cima marcado como `[TOPE RAM]`.

### 5.3.2. Captura 2. Búsqueda y modificación de datos o prioridades de procesos

```
ID Proceso: 4
Nombre    : dev.exe
Prioridad : 8
----------------------------------------
Total de procesos: 5

========================================
   GESTOR DE PROCESOS (LISTA ENLAZADA)
========================================
1. Registrar proceso
2. Mostrar procesos
3. Buscar proceso por ID
4. Modificar prioridad de proceso
5. Ordenar procesos por prioridad (mayor a menor)
6. Volver al menu principal
Seleccione una opcion: 3

Ingrese ID de proceso a buscar: 3

Proceso encontrado -> Nombre: antivirus | Prioridad: 6

========================================
   GESTOR DE PROCESOS (LISTA ENLAZADA)
========================================
1. Registrar proceso
2. Mostrar procesos
3. Buscar proceso por ID
4. Modificar prioridad de proceso
5. Ordenar procesos por prioridad (mayor a menor)
6. Volver al menu principal
Seleccione una opcion: 4

Ingrese ID del proceso a modificar: 3
Proceso: antivirus
Prioridad actual: 6
Ingrese nueva prioridad: 9

Prioridad actualizada correctamente.
```

**Figura 19.** *Búsqueda del proceso 1002 y modificación de su prioridad de 3 a 9.*

```
==========================================
   GESTOR DE PROCESOS (LISTA ENLAZADA)
==========================================
1. Registrar proceso
2. Mostrar procesos
3. Buscar proceso por ID
4. Modificar prioridad de proceso
5. Ordenar procesos por prioridad (mayor a menor)
6. Volver al menu principal
Seleccione una opcion: 5

Procesos ordenados por prioridad (mayor a menor).

==========================================
   GESTOR DE PROCESOS (LISTA ENLAZADA)
==========================================
1. Registrar proceso
2. Mostrar procesos
3. Buscar proceso por ID
4. Modificar prioridad de proceso
5. Ordenar procesos por prioridad (mayor a menor)
6. Volver al menu principal
Seleccione una opcion: 2

========== GESTOR DE PROCESOS (LISTA ENLAZADA) ==========
ID Proceso: 15
Nombre    : 55
Prioridad : 664
----------------------------------------
ID Proceso: 3
Nombre    : antivirus
Prioridad : 9
----------------------------------------
ID Proceso: 4
Nombre    : dev.exe
Prioridad : 8
----------------------------------------
ID Proceso: 2
Nombre    : excel.exe
Prioridad : 5
----------------------------------------
ID Proceso: 1
Nombre    : chrome.exe
Prioridad : 4
----------------------------------------
Total de procesos: 5
```

**Figura 20.** *Ordenamiento de procesos por prioridad de mayor a menor.*

*Nota.* Después del ordenamiento, word.exe (prioridad 9) pasa a ser el primer nodo de la lista.

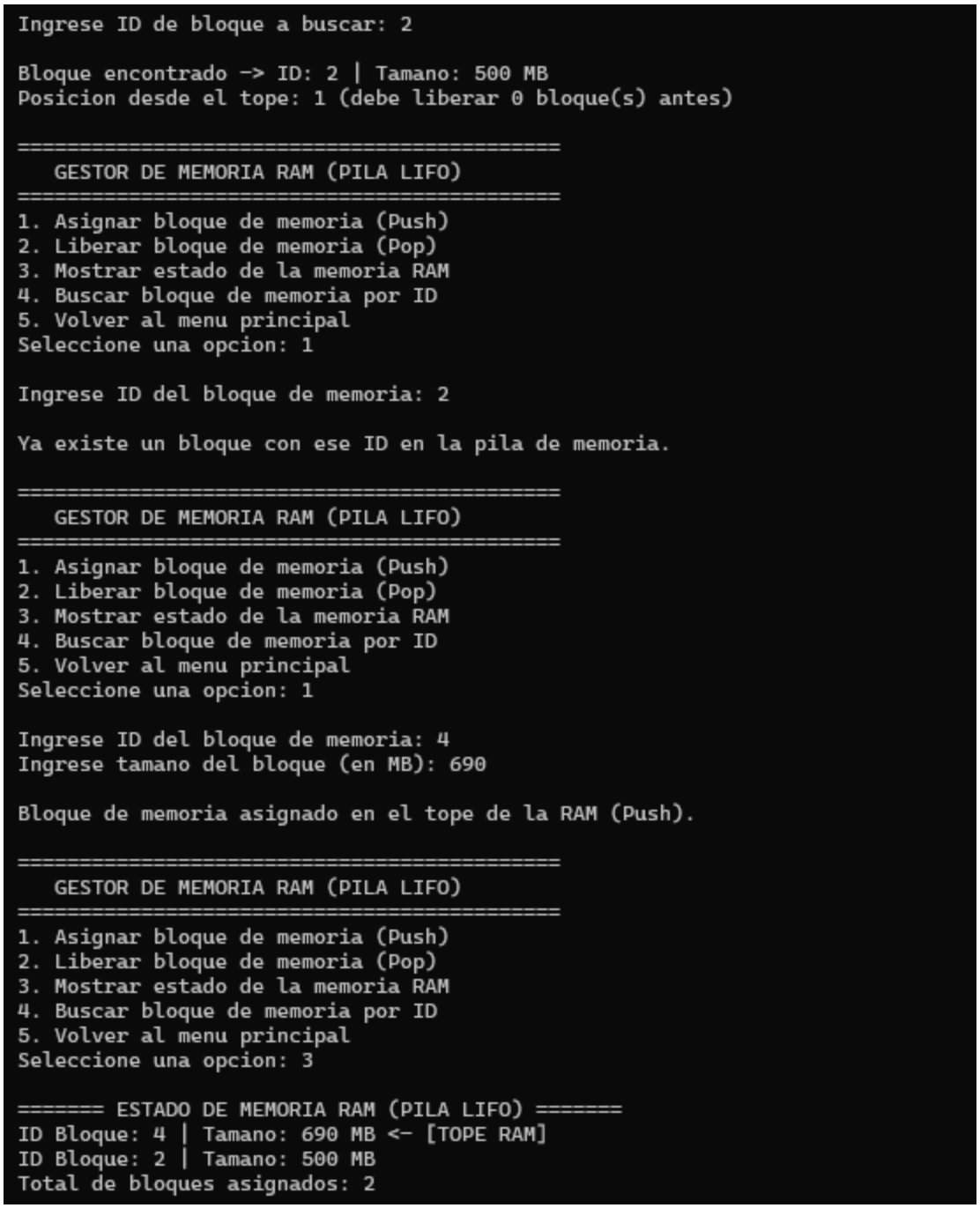



**Figura 21.** *Búsqueda de un bloque y liberación del tope de la pila (pop).*

*Nota.* El bloque 101 está en la posición 3; el pop libera primero el bloque 103 (LIFO).

### 5.3.3. Captura 3. Ejecución y despacho en CPU respetando el orden de llegada

```
Seleccione una opcion: 3

==========================================
   PLANIFICADOR DE CPU (COLA FIFO)
==========================================
1. Encolar tarea en CPU
2. Ejecutar tarea en CPU (Desencolar)
3. Mostrar cola de tareas de CPU
4. Modificar tarea por ID de proceso
5. Buscar tarea por ID
6. Volver al menu principal
Seleccione una opcion: 1

Ingrese ID de la tarea CPU: 2
Ingrese ID del proceso asociado: 1000
Ingrese descripcion de la tarea: MIL

Tarea registrada en la cola del Planificador de CPU (Encolar).

==========================================
   PLANIFICADOR DE CPU (COLA FIFO)
==========================================
1. Encolar tarea en CPU
2. Ejecutar tarea en CPU (Desencolar)
3. Mostrar cola de tareas de CPU
4. Modificar tarea por ID de proceso
5. Buscar tarea por ID
6. Volver al menu principal
Seleccione una opcion: 1

Ingrese ID de la tarea CPU: 4
Ingrese ID del proceso asociado: 2000
Ingrese descripcion de la tarea: DOS MIL

Tarea registrada en la cola del Planificador de CPU (Encolar).

==========================================
   PLANIFICADOR DE CPU (COLA FIFO)
==========================================
1. Encolar tarea en CPU
2. Ejecutar tarea en CPU (Desencolar)
3. Mostrar cola de tareas de CPU
4. Modificar tarea por ID de proceso
5. Buscar tarea por ID
6. Volver al menu principal
Seleccione una opcion: 3

========== PLANIFICADOR DE CPU (COLA FIFO) ==========
ID Tarea: 2 | ID Proceso: 1000 | Descripcion: MIL <- [SIGUIENTE EN CPU]
ID Tarea: 4 | ID Proceso: 2000 | Descripcion: DOS MIL
Total de tareas pendientes: 2
```

**Figura 22.** *Encolamiento de tres tareas en el Planificador de CPU.*




**Figura 23.** *Búsqueda, ejecución y modificación de tareas en la cola de CPU.*

*Nota.* Se ejecuta primero la tarea 10, que fue la primera en llegar (FIFO); la tarea 11 pasa a ser la siguiente en CPU.

### 5.3.4. Captura 4. Manejo de errores y validación de entradas

Se probaron entradas incorrectas para comprobar que el sistema no se bloquea: letras en campos numéricos, IDs repetidos, textos vacíos, opciones fuera de rango y extracciones sobre estructuras vacías. En todos los casos se muestra un mensaje y el programa continúa funcionando.

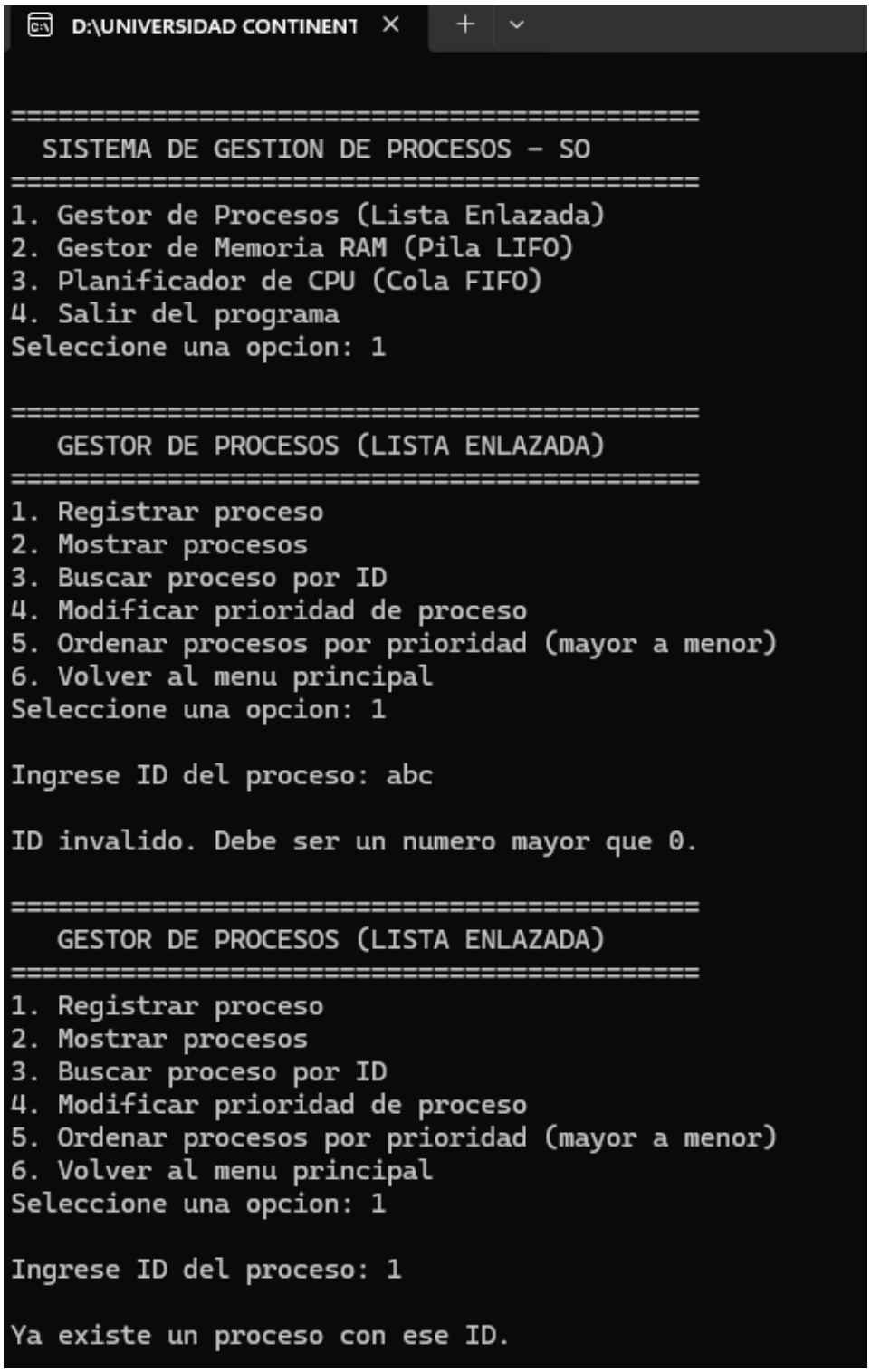
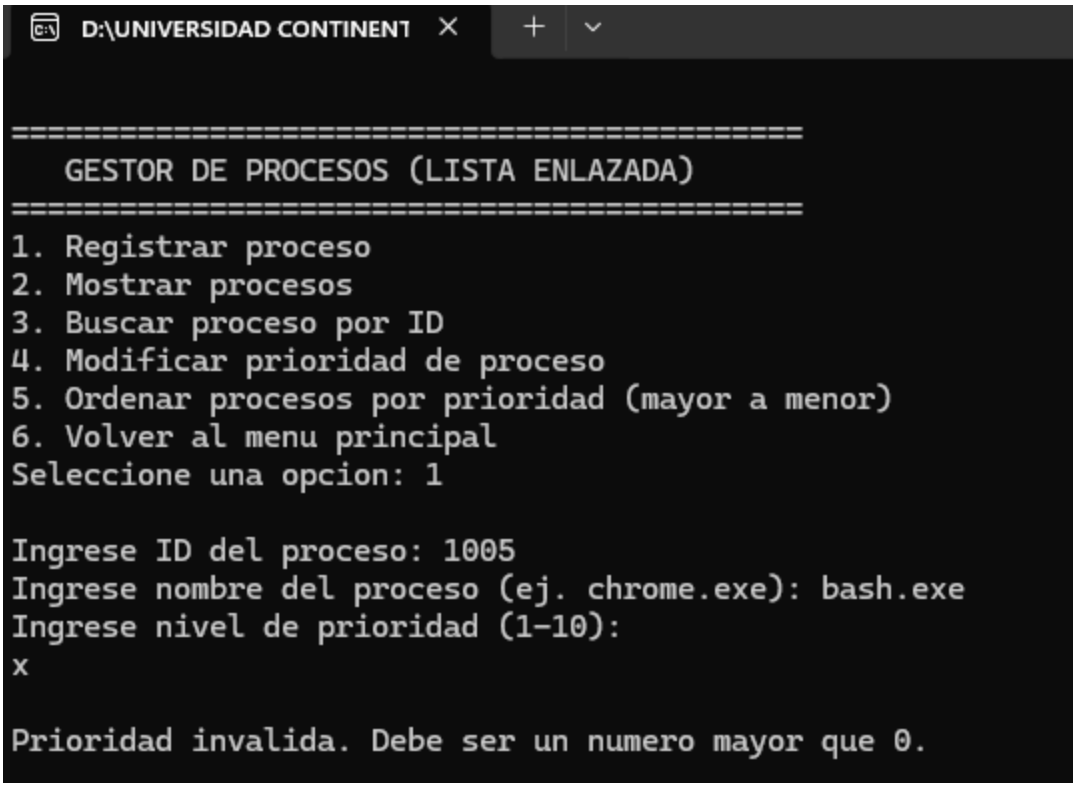



**Figura 24.** *Validación de ID no numérico, ID repetido y texto vacío (parte 1).*


D:\UNIVERSIDAD CONTINENT
==========
GESTOR DE PROCESOS (LISTA ENLAZADA)
==========
1. Registrar proceso
2. Mostrar procesos
3. Buscar proceso por ID
4. Modificar prioridad de proceso
5. Ordenar procesos por prioridad (mayor a menor)
6. Volver al menu principal
Seleccione una opcion: 1
Ingrese ID del proceso: 1005
Ingrese nombre del proceso (ej. chrome.exe): bash.exe
Ingrese nivel de prioridad (1-10):
x
Prioridad invalida. Debe ser un numero mayor que 0.

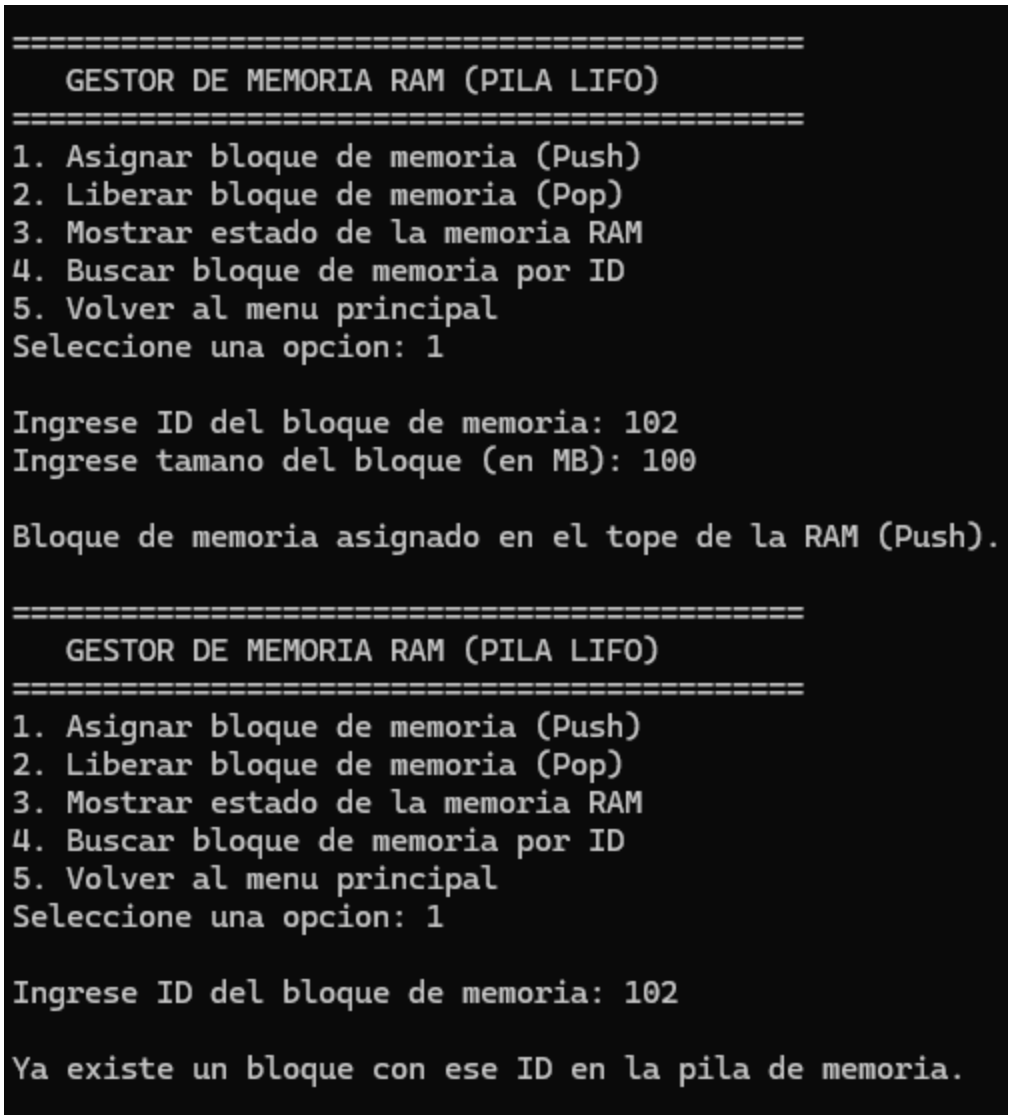



**Figura 25.** *Validación de prioridad no numérica, bloque repetido y opción no válida (parte 2).*

```
==========================================
   GESTOR DE MEMORIA RAM (PILA LIFO)
==========================================
1. Asignar bloque de memoria (Push)
2. Liberar bloque de memoria (Pop)
3. Mostrar estado de la memoria RAM
4. Buscar bloque de memoria por ID
5. Volver al menu principal
Seleccione una opcion: 2

Bloque de memoria liberado del tope (Pop) -> ID: 2 | Tamano: 300 MB

==========================================
   GESTOR DE MEMORIA RAM (PILA LIFO)
==========================================
1. Asignar bloque de memoria (Push)
2. Liberar bloque de memoria (Pop)
3. Mostrar estado de la memoria RAM
4. Buscar bloque de memoria por ID
5. Volver al menu principal
Seleccione una opcion: 2

Bloque de memoria liberado del tope (Pop) -> ID: 1 | Tamano: 200 MB

==========================================
   GESTOR DE MEMORIA RAM (PILA LIFO)
==========================================
1. Asignar bloque de memoria (Push)
2. Liberar bloque de memoria (Pop)
3. Mostrar estado de la memoria RAM
4. Buscar bloque de memoria por ID
5. Volver al menu principal
Seleccione una opcion: 2

La pila de memoria RAM esta vacia.
```

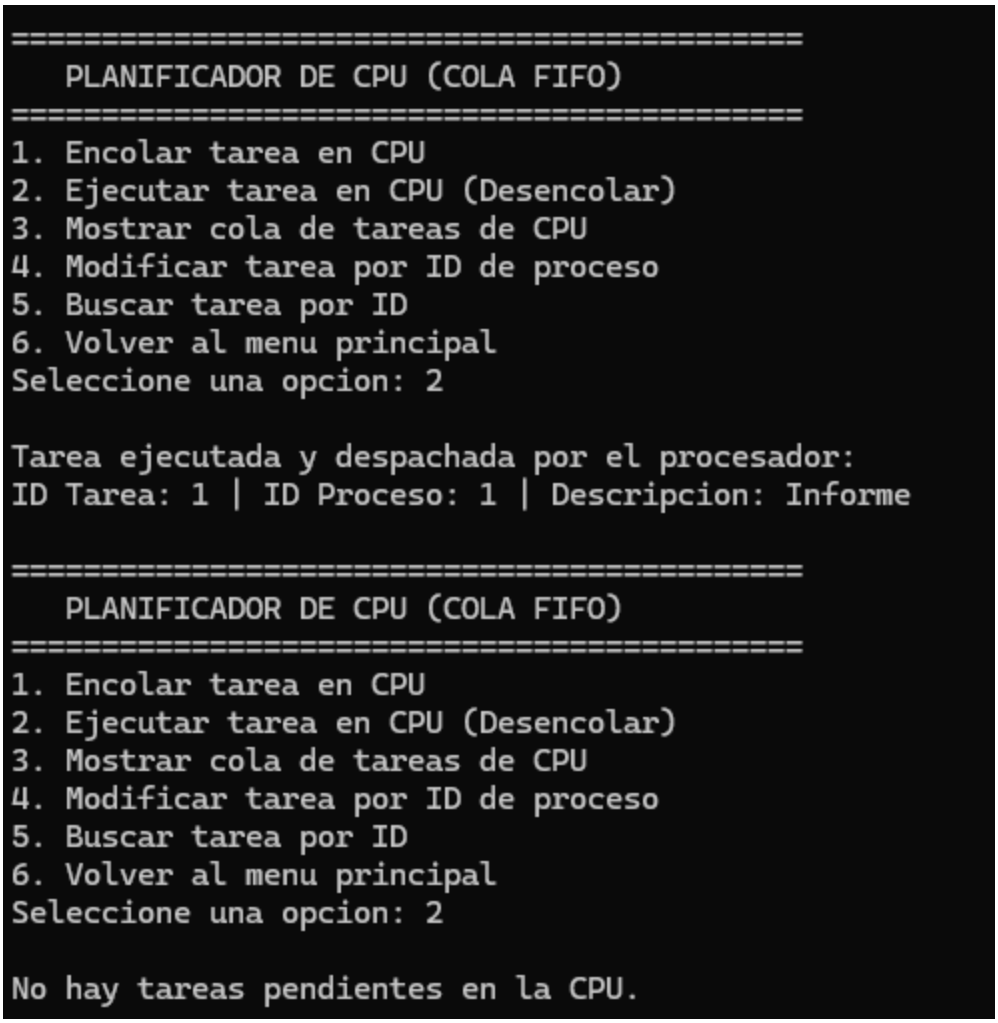



**Figura 26.** *Extracciones sucesivas hasta vaciar la pila de memoria y la cola de CPU.*

*Nota.* Al intentar extraer de una estructura vacía se muestra un aviso y no se accede a memoria liberada.

### 5.3.5. Captura 5. Guardado y recuperación del estado desde archivo

Para comprobar la persistencia se cerró el programa con la opción Salir y se volvió a ejecutar. Sin registrar datos nuevos, los tres módulos mostraron la información de la sesión anterior, incluido el orden por prioridad y la tarea modificada.

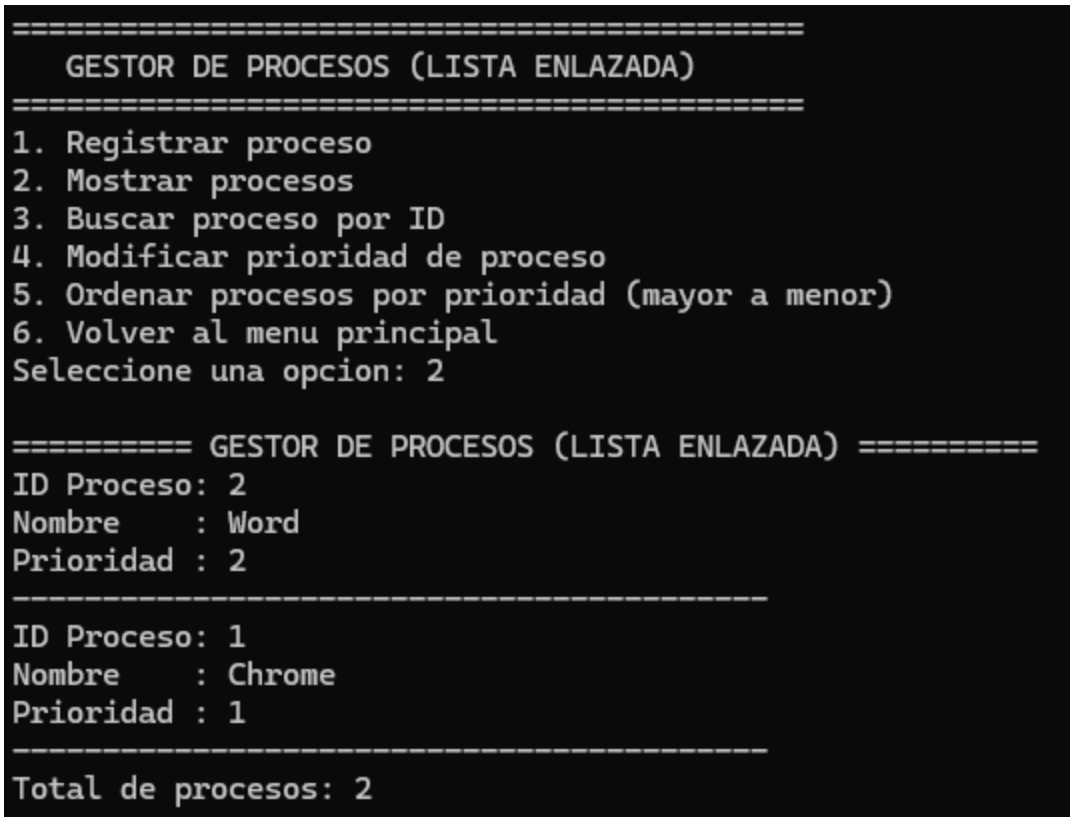



**Figura 27.** *Recuperación de procesos y memoria al reiniciar el programa (parte 1).*

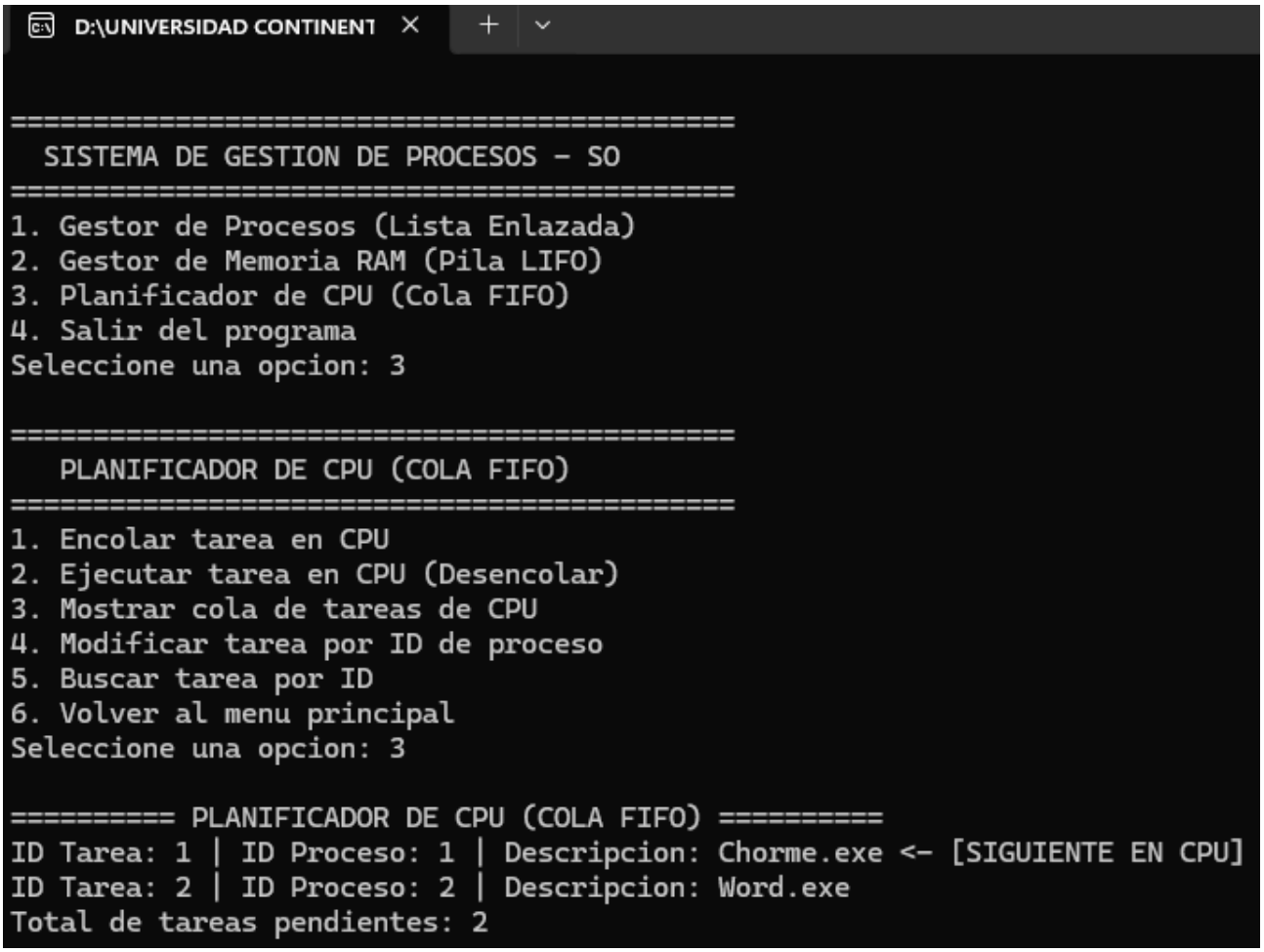


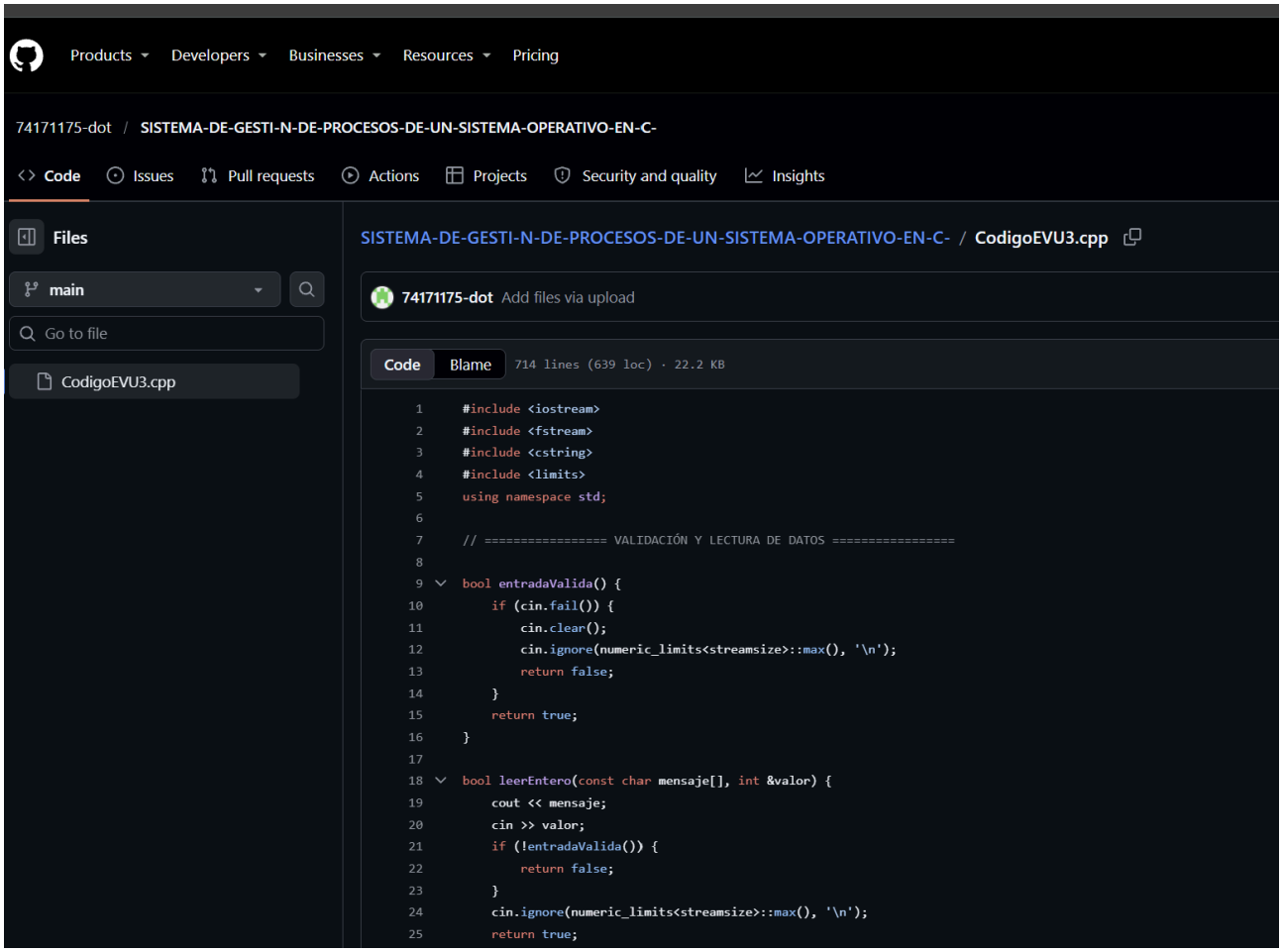

**Figura 28.** *Recuperación de la cola de CPU al reiniciar el programa (parte 2).*

# 6. GESTIÓN DEL TRABAJO EN EQUIPO Y CONTROL DE VERSIONES

El proyecto se desarrolló en un equipo de tres integrantes. Para evidenciar la participación de todos se utilizó un repositorio Git con historial de commits, una herramienta de gestión de tareas y actas de reunión donde se registraron los acuerdos y la distribución de responsabilidades. Cada integrante lideró un módulo, pero todos participaron en la revisión del código, la integración y la documentación. La comunicación se organizó mediante reuniones periódicas y un grupo de mensajería.

## 6.1. Evidencia del repositorio Git

https://github.com/74171175-dot/SISTEMA-DE-GESTI-N-DE-PROCESOS-DE-UN-SISTEMA-OPERATIVO-EN-C-




## 6.2. Matriz de roles y responsabilidades

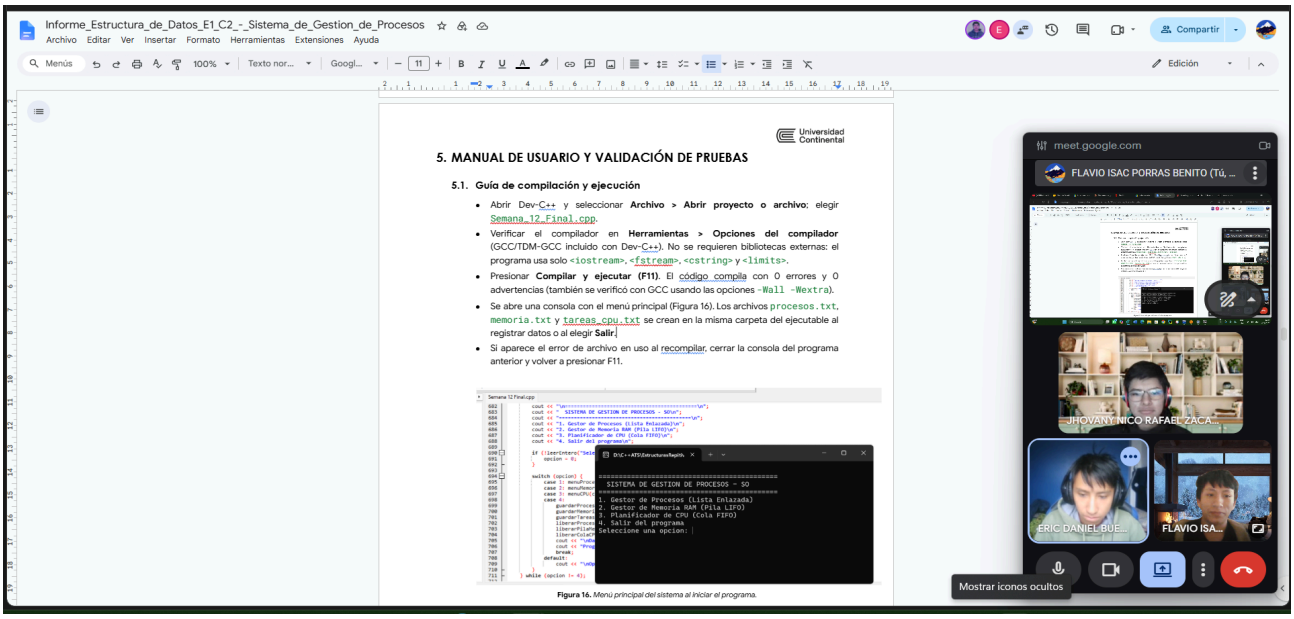
**Tabla 7.** *Matriz de roles y responsabilidades*

| Integrante (nombre y código) | Rol asumido | Módulo / tareas desarrolladas |
|---|---|---|
| Rafael Zacarias, Jhovany Nico | Líder de diseño | Main, Persistencia (archivos), Gestión de entrada/validaciones. |

| Integrante (nombre y código) | Rol asumido | Módulo / tareas desarrolladas |
|---|---|---|
| Porras Benito, Flavio Isac | Desarrollador Lista | Módulo de Gestor de Procesos (Listas enlazadas, ordenamiento). |
| Mayor Marticorena, Leonel Eduardo | Desarrollador Pilas | Módulo de memoria RAM (Stack LIFO). |
| Buendia de la Rosa, Eric Daniel | Desarrollador Colas | Módulo de Planificador de CPU (Que FIFO). |

## 6.3. Actas de reunión y seguimiento

**Tabla 8.** *Registro de reunión*



# 7. CONCLUSIONES Y RECOMENDACIONES

## 7.1. Conclusiones

- Se desarrolló un sistema en C++ que integra tres estructuras dinámicas lineales implementadas desde cero —lista enlazada simple, pila LIFO y cola FIFO— aplicadas a la gestión de procesos, memoria y CPU de un sistema operativo, cumpliendo el objetivo general.
- La lista enlazada permitió registrar, buscar, modificar y ordenar procesos sin fijar un tamaño máximo; las pruebas confirmaron la inserción al final y el ordenamiento correcto por prioridad (Figuras 17–20).
- La pila representó adecuadamente la asignación y liberación de memoria: en las pruebas, el último bloque asignado (103) fue el primero en liberarse, respetando el principio LIFO.
- La cola permitió administrar las tareas de CPU en orden de llegada: la primera tarea encolada (10) fue la primera en ejecutarse, respetando el principio FIFO.
- La persistencia con `<fstream>` conservó el estado completo del sistema entre ejecuciones, incluido el orden de los nodos.
- Las funciones de validación evitaron bloqueos ante letras en campos numéricos, textos vacíos, IDs repetidos y extracciones sobre estructuras vacías.
- La liberación de memoria con `delete` al salir del programa evita fugas de memoria.

## 7.2. Dificultades y soluciones aplicadas

**Tabla 9.** *Dificultades técnicas y soluciones*

| Dificultad | Solución aplicada |
|---|---|
| Mezcla de `cin >>` y `cin.getline()`: el salto de línea pendiente hacía que se saltara la lectura del texto. | Se agregó `cin.ignore(numeric_limits<streamsize>::max(), '\n')` dentro de `leerEntero()`. |
| Ingreso de letras en campos numéricos: provocaba ciclos infinitos en los menús. | `entradaValida()` limpia el estado con `cin.clear()` y descarta la entrada incorrecta. |
| Textos más largos que el arreglo `char`. | `leerTexto()` limpia el estado, descarta el sobrante y avisa que el texto fue recortado. |
| IDs repetidos en las tres estructuras. | Se validan con `buscarProceso()`, `buscarBloqueMemoria()` y `buscarTarea()` antes de insertar. |
| Reserva de memoria con datos inválidos. | Los datos se leen y validan en variables locales antes de crear el nodo con `new`. |
| Puntero `final` colgante al vaciar la cola. | Al ejecutar la última tarea se asigna `cpu.final = NULL`. |

| Dificultad | Solución aplicada |
|---|---|
| Mezcla de `cin >>` y `cin.getline()`: el salto de línea pendiente hacía que se saltara la lectura del texto. | Se agregó `cin.ignore(numeric_limits<streamsize>::max(), '\n')` dentro de `leerEntero()`. |
| Ingreso de letras en campos numéricos: provocaba ciclos infinitos en los menús. | `entradaValida()` limpia el estado con `cin.clear()` y descarta la entrada incorrecta. |
| Textos más largos que el arreglo `char`. | `leerTexto()` limpia el estado, descarta el sobrante y avisa que el texto fue recortado. |
| IDs repetidos en las tres estructuras. | Se validan con `buscarProceso()`, `buscarBloqueMemoria()` y `buscarTarea()` antes de insertar. |
| Lectura de archivos con textos que contienen espacios. | Formato de un campo por línea; `ignore()` antes de `getline()`. |

## 7.3. Recomendaciones

- Implementar una cola de prioridad en el planificador, modificando `encolarTarea()` para insertar cada nodo según la prioridad del proceso asociado.
- Agregar la eliminación individual de procesos en la lista enlazada (al inicio, en medio y al final), según el pseudocódigo de la Figura 9.
- Validar que el ID de proceso de una tarea exista en la lista de procesos antes de encolar, para relacionar los módulos.
- Mantener un puntero al último nodo de la lista para que la inserción al final sea O(1).
- Reemplazar el ordenamiento burbuja por un algoritmo más eficiente (por ejemplo, *mergesort* sobre listas) si la cantidad de procesos crece.
- Separar el código en archivos `.h` y `.cpp` por módulo y usar `std::string` para eliminar el límite fijo de caracteres.

# 8. REFERENCIAS BIBLIOGRÁFICAS


Cairó, O., y Guardati, S. (2006). *Estructuras de datos* (3.ª ed.). McGraw-Hill.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., y Stein, C. (2009). *Introduction to algorithms* (3.ª ed.). MIT Press.

cppreference.com. (s. f.). *std::basic_fstream*. https://en.cppreference.com/w/cpp/io/basic_fstream

cppreference.com. (s. f.). *std::basic_istream::ignore*. https://en.cppreference.com/w/cpp/io/basic_istream/ignore

Joyanes Aguilar, L., y Zahonero Martínez, I. (2007). *Estructuras de datos en C++*. McGraw-Hill.

Silberschatz, A., Galvin, P. B., y Gagne, G. (2018). *Operating system concepts* (10.ª ed.). Wiley.

# 9. ANEXOS

## Anexo A. Casos de prueba ejecutados

**Tabla 10.** *Casos de prueba y resultados obtenidos*

| N.º | Caso de prueba | Entrada | Resultado esperado | Resultado |
|---|---|---|---|---|
| 1 | Registrar proceso válido | 1001, chrome.exe, 5 | Proceso registrado | Cumple |
| 2 | ID no numérico | abc | "ID invalido..." | Cumple |
| 3 | ID repetido | 1001 | "Ya existe un proceso con ese ID." | Cumple |
| 4 | Nombre vacío | (Enter) | "El texto no puede estar vacio." | Cumple |
| 5 | Prioridad no numérica | x | "Prioridad invalida..." | Cumple |
| 6 | Ordenar por prioridad | 5 / 9 / 8 | Orden 9, 8, 5 | Cumple |
| 7 | Push de tres bloques | 101, 102, 103 | 103 en la cima | Cumple |
| 8 | Bloque repetido | 102 | "Ya existe un bloque..." | Cumple |
| 9 | Pop sobre pila vacía | opción 2.2 | "La pila ... esta vacia." | Cumple |
| 10 | Ejecutar tarea (FIFO) | opción 3.2 | Sale la tarea 10 | Cumple |
| 11 | Ejecutar sobre cola vacía | opción 3.2 | "No hay tareas pendientes..." | Cumple |
| 12 | Opción fuera de rango | 9 | "Opcion no valida..." | Cumple |
| 13 | Persistencia al reiniciar | Salir y volver a ejecutar | Datos recuperados | Cumple |

## Anexo B. Correspondencia entre funciones y líneas del código fuente

**Tabla 11.** *Ubicación de los bloques de código*

| Bloque del código | Funciones | Líneas |
|---|---|---|
| Validación de datos | `entradaValida()`, `leerEntero()`, `leerTexto()` | 9–42 |
| Registros y nodos | `Proceso`, `NodoLista`, `BloqueMemoria`, `NodoPila`, `TareaCPU`, `NodoCola`, `PlanificadorCPU` | 46–84 |
| Persistencia | `guardar*()` / `cargar*()` de procesos, memoria y tareas | 88–197 |
| Módulo 1: lista enlazada | `buscarProceso()` ... `liberarProcesos()` | 201–336 |
| Módulo 2: pila | `buscarBloqueMemoria()` ... `liberarPilaMemoria()` | 340–434 |
| Módulo 3: cola | `buscarTarea()` ... `liberarColaCPU()` | 438–581 |
| Menús | `menuProcesos()`, `menuMemoria()`, `menuCPU()` | 585–665 |
| Función principal | `main()` | 669–714 |

**Anexo C. Archivos entregados**

- `Semana_12_Final.cpp` — código fuente completo del sistema.
- `procesos.txt`, `memoria.txt`, `tareas_cpu.txt` — archivos de persistencia de ejemplo.
- Presentación de la sustentación y manual de usuario.